Fast and Accurate Statistical Simulation of Shared-Memory Applications on Multicore Systems

Fan Jiang, *Student Member, IEEE*, Rafael K. V. Maeda, *Student Member, IEEE*, Jun Feng, *Student Member, IEEE*, Shixi Chen, *Student Member, IEEE*, Lin Chen, *Student Member, IEEE*, Xiao Li, *Student Member, IEEE*, and Jiang Xu, *Member, IEEE*

Abstract—Detailed cycle-accurate simulation of multicore systems is naturally slow. Statistical simulation is one alternative that permits trading off simulation speed for accuracy. However, there is a lack of effective memory locality models for multicore applications. Hence, existing statistical simulators neglect data-sharing between threads. Additionally, the standard method to speed up statistical simulations is to blindly reduce the trace length to be synthesized. While this gives good control over the speedup, it leaves the simulation error unbounded. In this work, we introduce a novel statistical simulation methodology for exploration of shared-memory multicore systems. It includes a new **sha**ring-**lo**cality **m**odel (*Shalom*) that captures and reproduces data-sharing in multithread applications. Furthermore, we propose a method to bound the simulation error for a particular metric while maximizing speedup. The technique works by monitoring the convergence of the statistical synthesis. It is referred to as **con**vergence-**de**termi**n**istic **s**imulation (*Condens*). The combination of *Shalom* and *Condens* is around 130x faster than cycle-accurate simulations with reasonable accuracy loss. Our approach is also 5x faster than state-of-the-art sampling simulation under the same accuracy level. Compared to previous statistical simulators ignoring sharing, our approach is 2x more accurate for performance metrics and 8x more accurate for cache miss estimations.

Index Terms—Statistical simulation, reuse distance, performance modeling, simulation speedup, locality model, multicore system

1 Introduction

The traditional cycle-accurate simulation is naturally slow. On one hand, it offers great detail and accuracy. On the other hand, it becomes prohibitively slow when exploring large computing systems. This is especially a concern in early stages when the design space is huge. As a result, there have been a lot of investigations into methodologies that compromise detail and accuracy to gain simulation speed. Some examples are sampling simulation [1], [2], [3], [4], [5], parallelization techniques [6], [7], [8], [9], [10], statistical simulation [11], [12], [13], trace-driven simulation [14], [15], [16], and analytical estimations [17], [18], [19], [20]. The focus of this work is on statistical simulations.

A typical statistical simulation consists of two steps: profiling and synthesis. During profiling, we extract the essential characteristics of the application. During synthesis, we reproduce the behavior with statistical characteristics similar to those of the original. Then we use this synthesized behavior, instead of the original application, to feed the architecture simulator. One of the advantages of using the proxy model is that it converges faster to the average application behavior, which permits us to simulate only a fraction of the total trace length. This trace reduction leads to a proportional simulation speedup while keeping some level of accuracy. Another advantage of using statistical simulation is the ability to protect proprietary codes for benchmarking [11], [21], to allow the creation of artificial benchmarks [22], and to facilitate explorations of heterogeneous systems [18].

Statistical simulation is typically orders of magnitude faster than traditional cycle-accurate simulations. Therefore, it is especially helpful in early design stages, when there are many design points to investigate. Statistical simulation is not intended to replace other techniques. Instead, it should be used to narrow down the design space, allowing designers to spend more time and resources in the top candidate architectures.

While there exist a reasonable number of statistical models for single thread applications [11], [12], [13], [23], [24], models that target multi-threaded applications are fewer. Among the existing models for multicore applications, most ignore data-sharing between threads [25], [26]. We demonstrate that ignoring sharing can lead to up to 73.9% miss-rate estimation error for last-level caches (LLCs). The primary reason previous models did not consider data-sharing is that it is difficult to obtain a sharing model that is architecture independent (i.e., the model should not be affected by the host computer during profiling). The second reason is that even though there exist models that could be used for profiling, not all can be implemented as synthesis [27], [28], [29].

- *The authors are with Microelectronics Thrust, Hong Kong University of Science and Technology, Hong Kong, and also with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong. E-mail: {fjiangad, rkvivasmaeda, jfengah, schenax, lchencs}@connect.ust.hk, , jiang.xu@ust.hk.*

Manuscript received 23 June 2021; revised 24 Dec. 2021; accepted 11 Jan. 2022. Date of publication 18 Jan. 2022; date of current version 24 Mar. 2022.
(Corresponding author: Jiang Xu.)
Recommended for acceptance by A. Bhatele.
Digital Object Identifier no. 10.1109/TPDS.2022.3143535

1045-9219 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

Another issue with existing statistical simulators is that the standard method used to leverage speedup is to blindly reduce the length of the trace to be synthesized [11], [12], [13], [25], [26], [30], [31]. We refer to this technique as speedup-deterministic simulation (*SDS*). This method provides good control over the speedup because the simulation time is roughly proportional to the trace length. The downside of *SDS* is that it leaves the simulation error unbounded, letting the user with no control over accuracy. Previous investigations overlooked this drawback because the simulation error for single-thread applications tends to increase smoothly when the trace length is reduced. However, our experiments show that the error in multi-thread applications is more chaotic, making large jumps when the length is reduced. Therefore, the lack of control over the error is more problematic for multi-threaded applications.

This work makes three *contributions*. The *first* is a new **sha**ring-**lo**cality **m**odel (*Shalom*) that can capture and reproduce the data-sharing of multithread applications. The *Shalom* is constructed from the observation that data sharing can be divided into two categories: sequential and concurrent. This separation permits us to reduce architecture dependence during profiling. The *second* contribution is a new simulation method that allows the user to bound the error for a particular metric while obtaining maximum speedup under the error budget. This technique works by monitoring the convergence of the statistical models, and we refer to it as **con**vergence-**de**termi**n**istic **s**imulation (*Condens*). Our results show that *Shalom* and *Condens* together are around 130x faster than cycle-accurate simulations, while achieving a performance evaluation error of 36%. Our approach is also around 5x faster than sampling simulation under the same accuracy level. We also achieve better accuracy for cache miss rates, compared to previous statistical methods. The *third contribution* is an open-source application/architecture design environment where we release the tools used in this work. It contains (1) the statistical simulator incorporating the innovations in this work, and (2) a cycle-accurate (CA) simulator, which is built with QEMU [32] as the front-end. The back-end of both the statistical and the CA simulator are the same. It is a modified version of the RUBY memory simulator [33] with in-house models for the interconnect, CPU, and other peripherals [34].

This paper is organized as follows. We introduce the basic thread model we adopt for the applications in Section 2, followed by the profiling algorithms in Section 3, and the synthesis in Section 4. Finally, in Section 5, we detail the *Condens* mode.

2 Application Model

In this section, we introduce the application model that we assume for multithread applications and the high-level profiling algorithm.

2.1 Fork/Barrier/Join Thread Model

One of the most popular models for multithreaded application is the fork/join model. In this model, the execution flow of the application is divided into parallel sections (PSs), and transitions between them happen because of fork (creation of threads) and join (merging of threads) operations. In our work, we propose a slight modification to this model by also representing barriers. We refer to this as the fork/barrier/join model. In our model, barriers will also trigger transitions between PSs. We represent the fork/barrier/join model using what we refer as thread block section graph, as shown in Fig. 1. This example shows four (PSs), where each parallel section is composed of one or more TBSs. We define TBS as a sequential execution section that is executed by one physical thread. Note that this model puts in evidence the barrier synchronization which would not exist in the traditional fork/join model. The edges in the TBS graph ($\rightarrow$) represent the flow dependency between each of the TBSs.

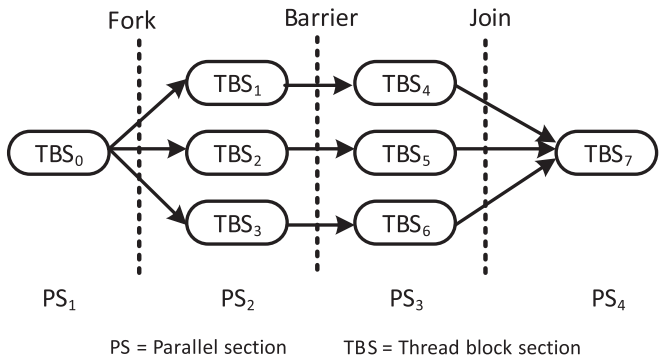


Fig. 1. Thread model based on fork/barrier/join.

This model helps us to separate the data-sharing into concurrent and sequential data-sharings. In simple words, sequential data-sharing happens when data in one PS is used in another PS, while concurrent sharing is the sharing between TBSs in the same PS. In Section 3, we revisit this terminology. While we are aware that the fork/barrier/join model is by no means a universal representation of all parallel applications, a significant subset including those based on Pthread and OpenMP library, such as SPLASH-3 [35], PARSEC [36], NAS Parallel Benchmarks (NPB) [37], can fit the model well. Hence, we will base our discussion on this model for the rest of the paper.

2.2 Application Profiling

In this section, we describe the profiling of the application flow. We profile the application during runtime with the goal of obtaining a TBS graph, similar to Fig. 1, as well as the profiling information for each of the TBSs. There are two types of information we need to observe during profiling. One type is function calls to synchronizations (i.e., fork, join, and barrier). We refer to this type as synchronization trace. The second type are the actual memory traces (instructions and data).

The profiling is shown in Algorithm 1. We omit implementation specific details for the sake of clarity. We assume the existence of a graph data-structure (G), which has the function AddNode and AddEdge. The algorithm is fairly simple, with the major challenge being to keep the mapping between the physical thread and the TBS. This issue arises because the scope of a physical thread extends for various PSs while a TBS exist only within one PS. We can overcome this using a dictionary (*dict*) to map the current physical thread to the corresponding TBS. In the algorithm, we use the function AddMap to insert a new mapping pair and

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

FINDTBS to get the corresponding TBS given the physical thread ID (tid).

Algorithm 1. TBS Graph Profiling

```
1:  root ← CREATENEWTBS(1)
2:  ADDMAP (dict, master_thread.tid, root)
3:  ADDNODE (G, root)
4:  for each trace in Application do
5:      cur ← FINDTBS (dict, trace.tid)
6:      if type is memory trace then
7:          PROFILEMEMORY (cur, trace.mem)
8:      else
9:          cur.sync_info ← trace.sync_info
10:         new ← CREATENEWTBS (trace.ntbs)
11:         ADDEDGE (G, cur, new)
12:         UPDATETIDMAP (dict, cur, new)
13:     end if
14: end for
```

The algorithm starts by creating the root TBS of the graph to represent the master thread in the first PS (lines 1 to 3). Then for each trace received, we use the physical thread ID (tid) to find the corresponding TBS it belongs to (line 5). The trace can be of two types: synhronization or memory request. In the case of memory access, we profile the memory request, updating the information of the current TBS (*cur*, line 7). The details of the function PROFILEMEMORY are explained in Section 3.

The less common case is when the trace is a synchronization trace (fork/barrier/join). The lines from 9 to 12 shows a simplified version of the profiling. The general idea is explained as follows. First, we need to terminate the current *TBS* and mark what type of synchronization event happened at the end (line 9). We also store extra information required. For instance, in the case of a barrier, we keep a unique identifier for it and how many threads the barrier awaits. A new PS will only be created after all threads hit the barrier. For the case of a fork, we record how many threads are spawned and the TID of the spawned threads. In the second step, we create N new TBSs, line 10. Then, we assign the flow dependency between the current and the newly created TBS, line 11. And last, we remap the tid to the newly created TBS, shown in the algorithm by the function UPDATETIDMAP.

An example of detail we omit in the algorithm is that when we create a new TBS (line 10), we first should make sure that the TBS has not been created already. For instance, in the case of a join, there might be several calls to the function *join*, one for each thread being joined. But there should be only one TBS leaving the join.

2.3 Critical Sections

During profiling, we also inspect calls to locks and semaphores. However, we treat them slightly differently to synchronization barriers. Barriers create new parallel sections, while critical sections do not. For each TBS, we record how many critical sections are there, and how many times the TBS enter the critical section. We record a unique identifier for each lock being acquired/released and how many times that TBS access uses that particular lock. This unique identifier is, for example, the memory address in the case of a *pthread* lock. For semaphores, we also record their initial value and whether the TBS is calling *wait* or *signal*. During our evaluations, we only inspect calls to functions from the pthread library, we make no attempt in capturing hand-crafted constructs.

TABLE 1
Example of Influence of the Thread Interleavings on the RD

Scenarios	Resulting interleaving	b_1 dist
(a) Uniform interleaving	$a_1, b_1, a_2, b_2, a_3, b_3$	4
(b) A $\gg$ faster than B	$a_1, a_2, a_3, b_1, b_2, b_3$	2

Assume b_1 is accessed again after interleaving.

3 SHARING-AWARE PROFILING

In this section, we detail the profiling of memory requests (instruction and data). The goal of this profiling is to capture the memory locality for each TBS. Our locality profiling extends the traditional locality analysis by also capturing data-sharings.

3.1 Overview

One of the most common memory locality metrics for single thread applications is the reuse distance (RD) [38], [39]. RD is defined as the distance between two accesses to the same memory location. It has a value of infinity when a particular location is first touched. Profiling the RD for a single thread is relatively simple. We scan a sequence of memory requests and compute the RD for each request. The output of the profiling is the RD distribution. One of the main reasons for the popularity of RD is that it can be obtained almost independently of architecture.

For multicore applications, the RD calculation is more challenging because there are multiple streams of memory requests (one for each thread) and their interleaving will affect the RD. Take for example two threads, A and B, that touch the locations $\{a_1, a_2, a_3\}$ and $\{b_1, b_2, b_3\}$, respectively. In this case, a_3 and b_3 are the most recently used data for each of the threads. Table 1 shows two possible cases: (a) perfect uniform interleaving, and (b) thread A is infinitely faster than B. Assume b_1 is accessed again after interleaving. As we can see, the distance of the memory location b_1 can change from 4 to 2 respectively. In this example, for a cache of size $S = 4$, accessing the symbol b_1 would be reported as a miss for case (a) and a hit for case (b).

Since the architecture influences the exact interleaving of the threads, it will also affect the computation of the RD. There exist some variations of the RD locality to reduce this influence during profiling. However, for statistical simulation, we also need a synthesis algorithm to generate a trace that has a similar locality as the original. Not all of these models can be implemented as synthesis. Therefore, in this work, we propose a new alternative method referred to as sharing-locality model (*Shalom*). The goal is to obtain the memory locality for individual TBS minimizing interference from the others. Here, we explain the general idea behind our model, and in Section 3.3, we show the detailed algorithm.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Locality Counters and Buffers in Each *TBS*

Type	Description	Symbol
History buffers	Local access for a TBS	B_{loc}
	Sequential sharing buffer	B_{seq}
Histogram	Local reuse distance	H_{loc}
	Sequential sharing distance	H_{seq}
	Concurrent sharing distance	H_{con}

We use upperscript to denote to whom the buffer or counter belongs. For instance, B^C_{loc} reads the local history buffer from the TBS C.

Data and instruction have separate buffers/counters.

Shalom is calculated for each TBS individually, see Fig. 1. For each TBS we profile the sequence of requests issued by it. We classify each request into two: local and non-local. In general, the first time a TBS access a memory location is treated as a non-local request. Once the TBS has touched it, any future access by this TBS to the same location is treated as local. We further divide the non-local access into three types. The first type is sequential sharing. This happens when the non-local memory location has already been used in a previous PS. The second type is concurrent sharing. This is when the non-local memory access is present in another TBS in the same PS. The third type is what we refer to as private access. This is when no other TBS has ever touched the memory location. This type is the true cold-miss of the application.

Our profiling consists of calculating the RD for local and non-local access separately. The strength of this approach is that only the calculation of the RD for non-local accesses are vulnerable to the interleaving of the threads. Fortunately, non-local access represents only a fraction of the total accesses for most of the applications, as we show in section 3.4.

3.2 History Buffer and Counters

Our profiling relies on the calculation of the RD. The main data-structure needed is a history buffer, which holds the most recently used addresses. The operations we need to perform in this buffer are: (1) insert an item in the front (TOFRONT) and (2) calculate the distance of this item (DISTANCE). Since the RD is calculated based on unique addresses, the function TOFRONT should first remove the item if it is already present before inserting to the buffer. There are many ways to implement this data-structure [38], [39], [40], [41], [42], [43], [44], [45]. Using a tree-like data-structure [43], we can implement it with a space complexity of $O(M)$ and amortized time complexity of $O(logM)$, where M is the number of unique addresses.

Each TBS has one history buffer to store the local access, and we refer to it as B_{loc}. We keep three histograms for each TBS. One to count the RD of local accesses (H_{loc}), another for the sequential sharing distances (H_{seq}), and the last for concurrent accesses (H_{conc}). Also, we assume that for each PS there is history buffer available holding the locations accessed by all previous PSs, we refer to it as sequential history buffer, B_{seq}. Section 3.4 explains how the buffer B_{seq} is maintained. Table 2 provides a quick reference for the list of counters and buffers for each TBS.

3.3 Memory Trace Profiling

Algorithm 2 shows the details for *Shalom* profiling. For each memory request received we update the counters and buffers of the TBS issuing the request, $ctbs$. For simplicity, we use the term address to refer to the cache line address since all our calculations are based on cache line address.

Algorithm 2. Memory Profiling Using *Shalom*

```
1:  function PROFILEMEMORY (ctbs, addr)
2:    if addr in B_loc^ctbs then
3:      dist ← DISTANCE (B_loc^ctbs, addr)
4:      H_loc^ctbs[dist]++
5:    else if addr in B_seq then
6:      dist ← DISTANCE (B_seq, addr)
7:      H_seq^ctbs[dist]++
8:    else
9:      found = False
10:     for each tbs in parallel section do
11:       if addr in B_loc^tbs then
12:         dist ← DISTANCE (B_loc^tbs, addr)
13:         H_con^ctbs[tbs, dist]++
14:         found = True
15:         breakfor
16:       end if
17:     end for
18:     if found = False then H_loc^ctbs[∞]++
19:   end if
20:   TOFRONT (B_loc^ctbs, addr)
21: end Function
```

We start by checking if the address is present in the local buffers (line 2). If found, this is a local access, and we perform similar operations as in a single-core RD profiling; we calculate the RD and update the histogram (lines 3 and 4). The other possibility is the access to be a sequential sharing. In this case, we update the sequential sharing histogram with the distance that the address is found (lines 5 to 7). The concurrent sharings are handled from line 10 to 17. For concurrent sharing, we scan the local buffers of all TBSs in the same parallel section (lines 10 and 11). If the address is found, we update the concurrent histogram with the ID of the TBS in which the data has been found and with the distance (lines 12 and 13). The last case is when the data access is private to the TBS, line 18, which is treated as infinity for the local RD. At the end of the iteration, we set the address to the most recently used position (line 20). Future access to this given address now will be found in the local buffers. Fig. 2 shows the classification of the four types of accesses and their fractions based on our experiments with 9 applications from the SPLASH-3 benchmark [35].

Our algorithm makes $O(NT)$ calls to the functions *distance* and *tofront*, where T is the number of threads in a PS and N is the trace length. Therefore, we expect the time complexity of the algorithm to be $O(NTlogM)$. Meanwhile, our algorithm maintains $O(T)$ buffers, so the space complexity is $O(TM)$.

3.4 Sequential and Concurrent Sharing

The B_{seq} buffer is used to keep a record of the most recently used data from past PSs. In the first PS, this buffer is empty

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

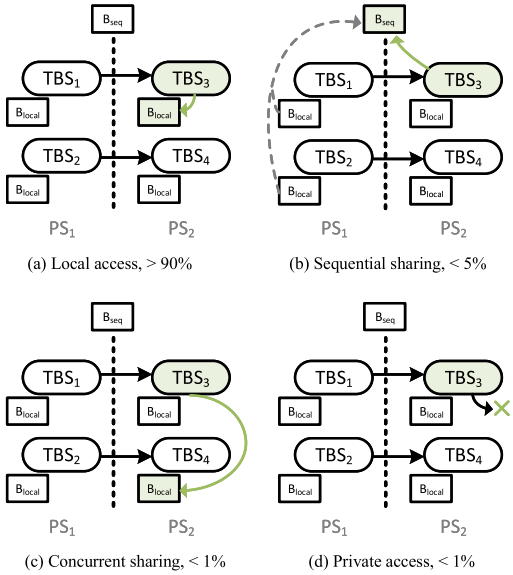


Fig. 2. The four types of accesses and their fractions.

as there is no previous data. For the following parallel section, the B_{seq} buffer holds the data accessed from previous PSs. This buffer is obtained at the end of every PS. We obtain B_{seq} to be used in PS $(t+1)$ by merging the local addresses touched by all TBSs in the previous PS (t). Table 3 shows an example with merging function (f) of uniform interleaving. There are two main reasons to do this merging. The first is to prevent the sequential buffer from being biased by the computer used to profile. The second reason is that it allows us to obtain consistent profiling results in different machines by masking the dependency from the architecture.

The merging will affect the RD computation, which might lead to misleading cache outcomes during simulation (i.e., false misses or hits). We define a false miss as the cache access that should have been a hit, but was simulated as a miss. Fig. 3a shows the proportion of false misses and hits for various cache sizes when we use the uniform interleaving. Applications with higher counts of threads tend to have higher false outcomes, as is the case for LU with 16-threads. The false outcomes are high for small caches, but they drop significantly for larger caches. For most of the applications, there are nearly no false outcome for caches larger than 512KB.

Fig. 3b, shows the average false outcome for 9 applications with ranging number of threads of 2 to 16. The bars represent false-misses and false-hits. The continuous line shows the net mismatch, false-miss subtracted by false-hit, for a 32KB cache. As we can see, the mismatch is as low as 2% for 2-thread applications and about 5% for 16-threads. This value means that for every sharing access there is an 5% chance for it to be a false miss in the L1 cache. In

TABLE 3
B_{seq} is a Function of All B_{loc} in the Previous PS.

TBS	$B_{loc}(t)$			$B_{seq}(t+1)$
A	a_1, a_2, a_3	$\rightarrow$		
B	b_1, b_2, b_3	$\rightarrow$	$f \rightarrow$	$a_1, b_1, c_1, a_2, b_2, c_2, a_3, b_3, c_3$
C	c_1, c_2, c_3	$\rightarrow$		

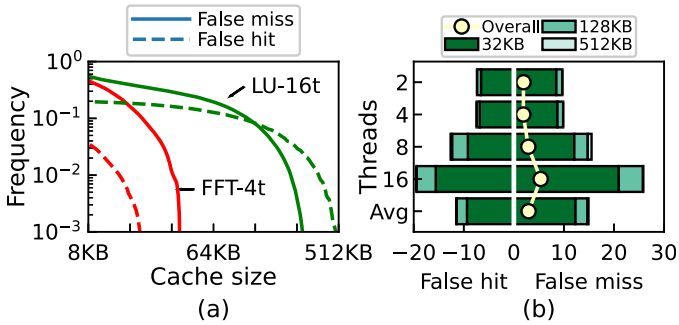


Fig. 3. Poportion of false miss/hit when using interleaving merging function. (a) Distribution of false miss/hit for FFT (4-threads) and LU (16-threads). (b) Average of false miss/hit rate for a cache size of 32KB, 128KB, and 512KB.

addition to this, our experiments show that the fraction of sharing access is less than 10% of the total requests. If we take this into consideration, the overall impact on the accuracy caused by the false-outcomes is even smaller.

It is substantially more challenging to remove the influence of the architecture for concurrent sharings. Therefore, their RD distributions are highly influenced by the architecture. Fortunately, the fraction of concurrent accesses is usually small. They account for less than 1% for multi-thread applications. We believe that using a different locality metric to capture the concurrent sharing could help reduce this dependence [29].

Even though the fraction of non-local accesses is small, they highly affect the accuracy of the simulation. As we will see later, models without sharing-locality are 3X less accurate than our proposed model. Two major issues arise when ignoring the sharings. First, it will lead to an increased number of cold misses. Second, there will be a similar increase in the application memory footprint. Therefore, capacity misses and conflict misses are going to be equally accentuated. For parallel applications, these source of errors are amplified proportionally to the number of threads.

4 Sharing-Aware Synthesis

The objective of the synthesis is to reproduce the application behavior. Including the application flow and the memory trace for each of the TBSs. This is the reverse process of profiling and therefore will be discussed only briefly.

4.1 Application Flow

The synthesis should reproduce the flow in the TBS graph. This requires traversing the graph and treating each TBS as a task. We map each TBS to one core during our simulation. The CPU dictates when the trace should be generated and issued, and it makes calls to our synthesis algorithm until we reach the end of the TBS. As in the profiling, the trace is of two types: memory trace or synchronization trace (fork/barrier/join). The memory trace generation is explained in the next section. Synchronizations are always issued at the end of the TBS execution using the information recorded during profiling. When they occur the simulation should also simulate the primitive. For example, for a fork, we spawn the next TBSs and schedule them in the cores available. During synthesis we also keep the sequential history

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

buffer for each PS, B_{seq}, which is maintained in the same way as in the profiling algorithm.

4.2 Memory Trace Synthesis

For each *TBS* we generate a trace length of N that equals the total number of requests. For each memory access, the synthesis algorithm begins by deciding whether it is an instruction or data access. This is be determined by randomly picking instruction with probability p proportional to the total number of instructions, $N_i/(N_i + N_D)$. The procedures for instruction and data are the same, but each follows its respective buffers and distribution profiles.

(A) Decide Type of Access. The first step is to decide the type of access that will occur: (*L*) local, (*S*) sequential sharing, or (*C*) concurrent sharing. In the real application, they occur in bursts. However, we assume that they are evenly distributed during the simulation because we are interested not in the detailed behavior, but rather, in the statistical average. Therefore, we simply choose from L, S, and C with a probability proportional to their frequency: p_l, p_s, and p_c. Once the type of access is defined, we perform a traditional address synthesis following their respective distribution. Each type is described below.

(B) Local Access. Here we proceed similar to the standard synthesis for a single-core RD. First, we generate a random RD, d, following the local RD distribution, H^{ctbs}_{loc}. Then, we find in the local buffer the address with distance, d, which will be reused. Then we set this address as the most-recently used by bringing it to the front of the buffer, $tofront(B^{ctbs}_{loc}, addr)$. In the case of an infinite reuse distance, $d = \infty$, we generate a new random address. We do so by randomly choosing an address uniformly distributed in the address space, $U(0, 4GB)$.

(C) Sequential Sharing. For sequential sharing, we reuse an address from the sequential history buffer, B_{seq}. The address is chosen for a random distance following the sequential distribution, $d \sim H^{ctbs}_{seq}$. Finally, we insert the address in the local buffer of the *TBS*, $insert(B^{ctbs}_{loc}, addr)$.

(D) Concurrent Sharing. Concurrent sharing requires the generation of two random numbers. The first is the corresponding *ID* of the *TBS* from which we will reuse the data. The second number corresponds to the reuse distance, d. They are obtained from the 2D histogram H^{tbs}_{conc}. Then we find the address in the local buffer of the target TBS, $addr = get_by_dist(B^{id}_{loc}, d)$. We finish by inserting the address in the local buffer of the current TBS, $insert(B^{tbs}_{loc}, addr)$.

We also need to determine the type of the request–read or write (*RW*)–and the size of the fetch. These choices are obtained following the frequency in which they happen for each type of access (local/sequential/concurrent).

4.3 Critical Sections Generation

During the generation of each TBS, we enter the critical section proportionally to the number of times they entered in profiling. And we generate the average number of memory requests for that critical section. We assume that critical sections are reached with the same average rate as in the original application for each TBS. The locality inside the critical regions follows the same procedure as explained in Section 4.2 for memory trace synthesis. However, their counters belong to the critical region as the locality inside critical regions tend to differ from the locality in non-critical regions.

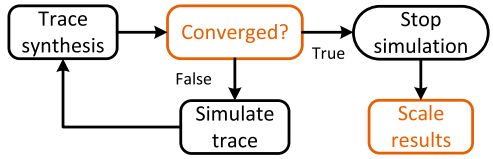


Fig. 4. Convergence-deterministic statistical simulation flow.

5 Speeding-Up Simulation

In this section, we discuss two techniques to speedup the statistical simulation.

5.1 Speedup-Deterministic Simulation

The common approach to speed up statistical simulation is what we refer to as speedup-deterministic simulation (*SDS*). In this simulation mode, during synthesis, instead of synthesizing a trace with the same length as the original application, we synthesize only a fraction. *SDS* offers good control over the speedup because the simulation time is roughly linear with the trace length. The drawback is that it leaves the users with no control over the simulation accuracy. This issue is often overlooked for single-core applications because they tend to have a smooth error increase when the trace length is reduced. However, our experiments show that the error in multi-thread applications is more chaotic, having big jumps for reduced trace length. Also, under the same reduction factor, multithread applications experience higher error than single-thread. Therefore, the lack of control over the error is more problematic for multi-threaded applications.

5.2 Convergence-Deterministic Speedup

In this work, we propose a simulation method that allows the user to bound the error for a particular metric while obtaining maximum speedup under the error budget. This technique works by monitoring the convergence of the statistical models, hence the name convergence-deterministic simulation (*Condens*). Fig. 4 shows the overall idea of *Condens*. We monitor the convergence of the statistical model and allow stopping the synthesis only after the target level is reached. This simulation mode offers better control over the accuracy since convergence has a direct relationship with accuracy. However, the simulation speedup becomes an outcome rather than an input. For the same convergence-level, different applications will experience different speedups.

A practical approach to estimate the convergence is to observe a performance metric m_{obs} and compare it to a golden reference value m_{ref}, as in equation (1). Ideally, we would use a system metric, such as execution time or IPC. However, this would require us to have a golden reference value prior to the simulation. Since they are architecture dependent, it is impossible to know them beforehand.

$$conv = |m_{obs} - m_{ref}| \tag{1}$$

Therefore, we propose estimating the convergence level using the memory locality. The metric we choose is the analytical miss-rate for fully-associative (FA) caches using the

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

LRU replacement policy. There are two reasons for this choice. The first is that we already capture the RD of the application during the profiling. The second reason is that the analytical miss rate depends only on the cache size, which can be obtained as input for the simulation. We start our discussion on how to obtain the golden miss rate reference value m_{ref}. For a given cache size, C_S, we obtain the golden miss rate by counting the fraction of RD larger than the cache-size, $m_{ref} = P[RD > Cs]$ using the RD distribution collected during profiling (H_{loc}). This value is obtained once for each TBS, before the synthesis.

To obtain the observed miss-rate m_{obs} we simulate a FA cache with the same size C_S. This cache is simulated in parallel and independent from the actual target architecture under evaluation. To avoid confusion, we refer to the FA cache implemented in the *Condens* as monitor cache. We simulate each memory reference created during synthesis in both the monitor cache and the target architecture. Then we use the reported miss rate by the monitor cache, m_{obs}, to estimate the convergence level following Equation (1).

Since a typical computer system has multiple levels of cache with different access latencies, we use multiple monitor caches matching in size with the ones in the target system. We use a weighted average across all levels, with weights proportional to their miss penalty, as shown in Equation (2).

$$conv_{ov} = \sum_{i=L1,L2,L3} \left(|m^i_{obs} - m^i_{ref}| \times w_i \right) \quad (2)$$

One key observation is that we can simulate the monitor FA cache in the statistical simulation with almost no cost by adding a few modifications in the synthesis algorithm described in Section 3.3. To simulate the monitor cache we have to maintain two counters: the total number of accesses, and the total misses. The total number of access is incremented for each local address generated. For the misses we first have to recall that to generate a memory request we first generate a random RD (rd, see step A in Section 3.3). Therefore, the total misses is incremented whenever $rd > C_S$. And the miss rate is obtained with $m_{obs} = total_misses/total_count$.

5.3 Scaling Performance Counters

When using any of the speedup techniques, we need to scale the final results to predict the actual performance of the system. In both *SDS* and *Condens*, we perform the speedup per TBS. In applications with a single TBS, scaling the results is trivial assuming that the average performance rates (e.g., IPC, and miss rates) are constant. For instance, we can obtain the actual execution time (T_{final}) using Equation (3), when the original application length is N_{tot} and the reported IPC is IPC_{sim}.

$$T_{final} = IPC^{-1}_{sim} \times N_{tot} \quad (3)$$

For multi-threaded applications, there could be several PSs with different number of TBSs each. We first treat the case for *SDS*, then later we discuss the *Condens* mode. In *SDS*, we know before-hand by how much each TBS will be reduced and all TBS experience the same reduction. At the end of each PS, we scale the results assuming that the results from the previous PSs do not affect the current average.

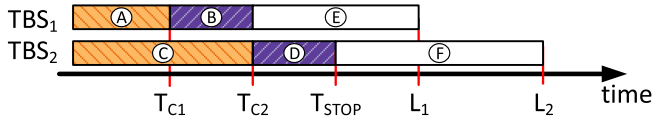


Fig. 5. Time to convergence of a parallel section with two $TBSs$.

Therefore, the total execution time of a parallel section is given by the total trace length of that parallel section, N_t, times the corresponding CPI (IPC_t^{-1}). We understand that, in fact, previous PSs might affect the current IPC average. However, considering their effect does not lead to a substantial accuracy improvement [4].

Scaling the performance results for *Condens* is slightly more complicated because TBSs in the same PS might converge at different times. Fig. 5 illustrates this with one PS and two TBSs. The horizontal axis shows the total trace length simulated for each of the TBSs. Here TBS_1 has total trace length L_1 while TBS_2 has L_2. We assume that the time for convergence for each of the TBSs are T_{C1} and T_{C2}, respectively.

Note that if we stopped the TBSs as soon as they converge, it will lead to a misleading resource contention. To guarantee a realistic contention between the two TBSs, we simulate them until all TBSs belonging to the same parallel section converge, marked in Fig. 5 as time T_{C2}. At this point we allow the smallest TBS to finish execution, and the reduction factor experienced by it is $r = L1/T_{C2}$. Then we continue to simulate TBS_2 until it face the same reduction factor. In this example, this happens to TBS_2 at time $T_{stop} = L2/r$. At this point we stop TBS_2 and the PS is finished. We then scale the results using the same approach as in the *SDS* and the simulation proceeds to the next PS. We can note that the actual simulation speedup obtained comes from the fact that we did not have to simulate the regions E and F.

6 Methodology

Our methodology consists of two steps. First, we profile the application to obtain the TBS graph and the locality profile for each TBS. Then, we use these models to drive the statistical simulation using our synthesis algorithm.

6.1 Profiling

We modify the QEMU emulator to implement our profiler. As previously mentioned, the interleaving of the threads during profiling might affect the final result. The host machine used for profiling and simulation is a 2-socket Intel Xeon E5-2687W, with a 48GB RAM, and CentOS 7.

We compile the applications at their recommended optimization levels. The target ISA chosen is x86_64. The ISA affects the memory locality of the application. However, we expect our technique to perform equally well for different ISAs. The input for the benchmarks is chosen to be the medium size. At the moment, our profiling is input specific. Therefore, we need one profiling for each input data-set chosen. Once the profile has been obtained, it can be reused as many times as needed to evaluate different architectures.

As mentioned in Section 3.2, the history buffer size used in profiling should ideally be $O(M)$ large, where M is the number of unique addresses in a trace. However, fitting $O(M)$ entries in memory is not practical for large values of M, so we must limit the history buffer to be C-entry large, where C is a

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
Simulated Target System

Compiler	GCC version 9.1.0, -O2
OS	QEMU system-call emulation mode
CPUs	Varying number of cores $\{1, 2, 4, 8, 16\}$ x86_64, 1-way, In-Order, 1-stage pipeline Frequency 2GHz
Cache	I/D-cache 32KB, 4-assoc Unified L2-cache 256KB, 8-assoc Unified L3-cache $2MB$, 16-assoc L1/L2 are private; L3 is shared Inclusive; write-back/write-allocate 64-bytes cache line, pseudo LRU.
Latencies (cycles)	L1=1 / L2=12 / L3=30 / Mem=158
Interconnect	Crossbar with 3-pipeline stages
Coherence protocol	L1/L2 are MSI and L3 is MOSI.

constant. Limiting the buffer size to C results in an approximation that any RD greater than C now is treated as infinity. This approximation may create false cold miss. This happens during trace synthesis when an infinite RD is met and thus a new address is generated, leading to a cold miss, while, in reality, the RD should have a large value greater than C and generate a reused address which does not cause a cold miss. Fortunately, the impact of this approximation is expected to be low if the number of cache lines of the target architecture under evaluation is smaller than C. Specifically, an access to the address with a large RD is more likely to be a cold miss since the cache line is likely to have been evicted. When a RD becomes greater than the number of cache lines, it is almost guaranteed to cause a cold miss. Therefore, storing the large RD's value or simply treating it as infinity in our model does not make much difference. In our profiling setting, we limit all history buffers to be $128K$-entry large and assume a cache line to be $64B$. This combination gives an accurate RD calculation for any caches smaller than $8MB$, which is adequate for our experiments.

6.2 Simulation Environment

In our work, we use two simulators, one statistical and the other cycle-accurate (CA). The CA simulator is built using QEMU as the front-end. The QEMU receives feedback from the architecture simulator to instruct which thread should be allowed to execute. During CA simulation, we set QEMU in the system-call emulation mode. The statistical simulator uses our synthesis algorithms as the front-end. Both the statistical simulator and the CA simulator have the same back-end, which is a multicore system simulator built on top of the RUBY memory simulator. We include several in-house models for interconnect, CPU, and other peripherals.

Table 4 summarizes the target architecture simulated. We consider the CA accurate simulator to be the golden reference.

6.3 Evaluations

We refer to the statistical simulation using our sharing-locality model as *Shalom*. First, we evaluate how *Shalom* stands compared to the baseline statistical simulation model that disregards sharing-locality [25], [26]. The latter is referred to as non-sharing model (*NSM*). Then, we compare our techniques with state-of-the-art sampling techniques for multithreaded applications. We refer to the sampling technique as *SAMP*. Unless specified otherwise, we use *Shalom* and *NSM* with full simulation length, and for *Condens*, we use a convergence level of 0.1.

We use a collection of 9 applications from the SPLASH-3 benchmark [35]. We exclude the variations of the same application (e.g., *ocean-contiguous* and *water-nsquared*). Each benchmark is simulated with various numbers of threads {1, 2, 4, 8, 16}, giving us, in total, 45 simulation points. We map one thread to each core of the target system.

7 Results

In this section, we compare *Shalom* and *Condens* with existing statistical simulation and sampling simulations.

7.1 Performance Accuracy

Fig. 6 shows the accuracy for several performance metrics. In this figure, we compare *NSM*, *Shalom*, and *Condens*. We use *Condens* to refer to the combination of (*Shalom* + *Condens*). *NSM* and *Shalom* are simulated with full simulation length while *Condens* is set to have a convergence level of 0.1. The top-row charts show the average error grouped per number of threads. The value *Avg* refers to the average across all 45 simulation points. The y-axes are the relative error for all plots except for L3 MPKI, which is the absolute error. The bottom-row charts report average error on the y-axes and correlation (Pearson's correlation coefficient) on the x-axes. The x-value of each marker is the correlation between CA and statistical simulation results with the same thread number, indicating how closely the model follows the original trend (a value closer to +1 means a higher correlation). There are five markers showing correlation and error for number of threads {1, 2, 4, 8, 16} plus one additional marker for the overall results. Therefore, there are in total six markers per technique. Points closer to the bottom-right corner are better.

Column (a) and (b) of Fig. 6 presents the results for execution time and the IPC, respectively. Another important metric is the relative application speedup, shown in column (c). We define the application speedup as the execution time for a single thread, T_1, divided by the execution time for the same application with N threads (i.e., $Speedup = T_1/T_N$). This is an important metric as it gives the scalability of the application. We compute the relative speedup for each benchmark for $N = \{2, 4, 8, 16\}$. Then, we compare the speedup reported by the statistical models (s_{stat}) and the actual application speedup obtained through CA simulation, s_{ca}. We define the speedup error as $error = |s_{stat} - s_{ca}|/s_{ca}$. By definition, the speedup for a single-thread application is 1, therefore the error is always zero. The last column, (d), shows the L3 cache misses per kilo instruction (MPKI) absolute error.

The overall observation is that the proposed techniques *Shalom* and *Condens* are superior to *NSM* for all metrics. Note that for single-thread applications our models perform as accurately as *NSM*, since there is no sharing. However,

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

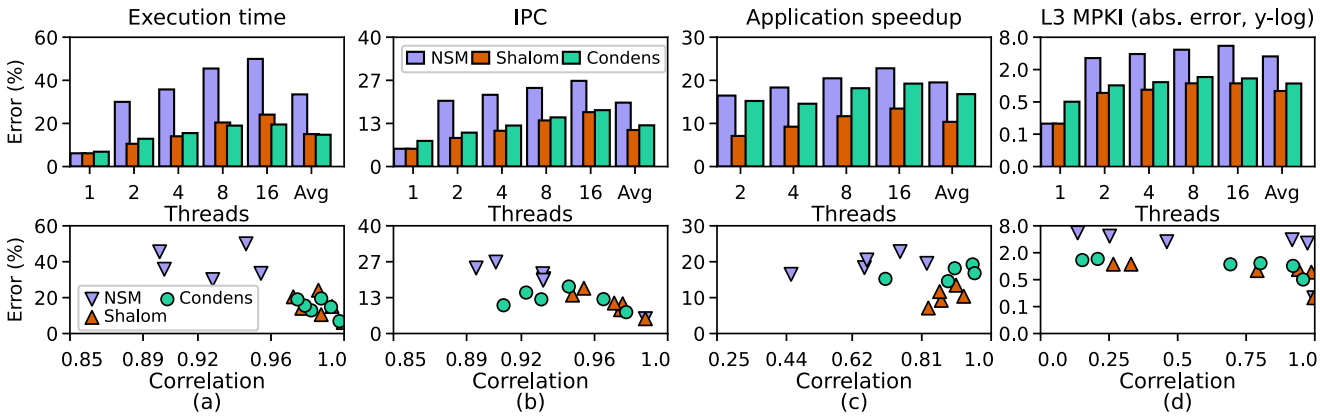


Fig. 6. Accuracy for various perfomrance metrics. Bottom figures show the error versus correlation. (a) Execution time. (b) IPC. (c) Relative application speedup. (d) Last-level cache misses-per-kilo instruction (MPKI), y-axis shows the absolute error in log-scale.

as the number of threads increases, *Shalom* and *Condens* become noticeably better. Our models help not only improve accuracy but also increase the correlation. Therefore, they better reproduce the performance trends among the benchmarks. This improvement comes from the more accurate memory locality models. Ignore sharing in *NSM* leads to an increased number of cold miss and memory footprint, meanwhile resulting in more capacity miss and conflict miss. These source of errors expand with increase in number of threads. The sharing-locality model *Shalom* proposed in this work allows us to capture sharings between threads, which improves the accuracy across all levels of cache. Take, for instance, the L3 MPKI in column (d). The MPKI error is reduced by more than 4x when using our sharing locality model. We discuss in more detail the LLC in section 7.2.

Note that for all models the simulation error increases for applications with higher counts of threads. The main reason is that it is difficult to match the actual interleaving of the application with the statistical simulation, which might result in misleading resource contention. And the higher the number of threads in the application, the more difficult it is to capture such behavior.

On average, both *Shalom* and *Condens* perform equally well in execution time estimation, with about 15% error, which are around 2x more accurate than *NSM*. For application speedup and IPC, *Condens* performs slightly worse than *Shalom*, but they are both more accurate than *NSM*. The IPC errors of *Shalom*, *Condens*, and *NSM* are 11.3%, 12.8%, and 19.8%, respectively. For application speedup, the errors of *Shalom*, *Condens*, and *NSM* are 10.4%, 16.8%, and 19.5%, respectively. For all metrics, *Shalom* has better correlation than *NSM* as we can see by the higher concentration of points in the bottom-right corner of the second-row charts.

Note that *Condens* has lower correlations and higher errors than *Shalom* for all metrics except for execution time. The reason is that *Condens* only simulates a fraction of the synthesized trace while *Shalom* uses a full-length trace. However, full simulation length does not necessarily result in better accuracy than reduced length, which can be seen in column (a). The errors of *Condens* are comparable to those of *Shalom* in the simulation of 1-thread, 2-thread, and 4-thread applications, and when the number of threads increases to 8 and 16, *Condens* even has lower errors than *Shalom*. At first glance, this seems counter-intuitive because we would expect the full simulation length to lead to the most accurate results. However, this is not always the case because the synthesis algorithm brings biases into the simulation. One example is that when we generate a new memory address, we randomly obtain it following a uniform distribution in the address space. However, the true distribution of the address is not uniform. Therefore, our locality synthesis will not match perfectly with some cache metrics, such as conflict misses. So, when we simulate the application for the full length, there is a higher chance of bias. By contrast, *Condens* stops as soon as the statistical models converge, which reduces the chances of biases occurring during synthesis.

7.2 Miss Rate Accuracy

The average miss rate error is shown in Table 5. Each row of this table shows a different cache level accuracy. We break down the average errors and group them by the number of threads. The last column is the average over all benchmarks. The observation is that for the first level of cache (data and instruction) all models are very accurate. Their average error is less than 1%. For the second level of cache, also private, the respective errors of the three models are comparable, with an average error of around 13% for the L2.

Conversely, for the last-level cache (LLC), *Shalom* is on average 8 times more accurate than *NSM*. This difference in

TABLE 5
Average Miss Rate Error (%) Breakdown Per Thread

Thread	Model	L1-D	L1-I	L2	L3
1	*NSM*	0.1	0.0	7.0	4.3
	Shalom	0.1	0.0	7.0	4.3
	Condens	0.1	0.0	14.6	9.7
4	*NSM*	0.4	0.0	15.5	64.1
	Shalom	0.5	0.0	13.6	7.2
	Condens	0.5	0.0	13.4	11.4
16	*NSM*	0.4	0.1	15.6	73.9
	Shalom	0.9	0.0	15.9	6.9
	Condens	1.0	0.0	13.6	7.3
Avg	*NSM*	0.4	0.0	13.8	53.9
	Shalom	0.5	0.0	12.6	6.6
	Condens	0.5	0.0	13.9	9.9

The bottom rows show the average over all benchmarks.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

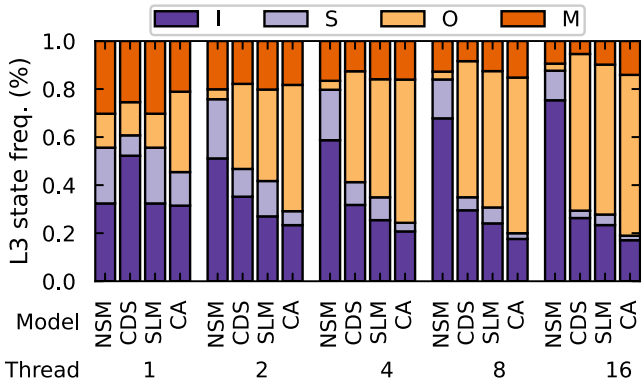


Fig. 7. L3 cache-line state breakdown per access. Breakdown obtained for the MOSI cache coherence. We use CDS to refer to *Condens*, and SLM to *Shalom*.

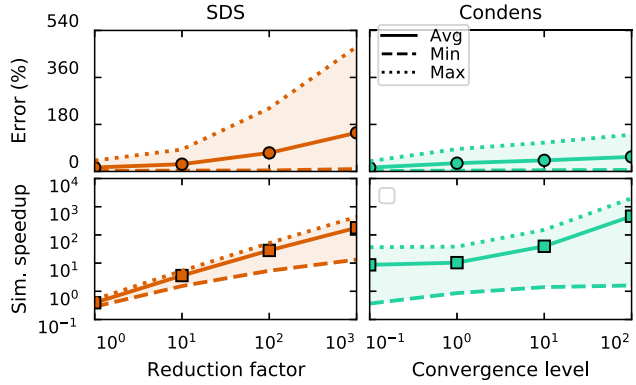


Fig. 8. Comparing speedup-deterministic simulation (*SDS*) and convergence-deterministic simulation (*Condens*).

accuracy is caused by the fact that *NSM* is not able to capture sharings between threads for multithreaded applications. Therefore, all data reused between parallel sections and between TBSs in the same parallel section are not well reproduced in the statistical simulation using *NSM*. In fact, most of the data-sharing between TBSs will be perceived as LLC misses. The miss rate error in the LLC reaches up to 73.9% for *NSM* for applications with 16 threads, while *Shalom* and *Condens* have errors noticeably lower, at 6.9% and 7.3%, respectively.

We also report the misses per kilo instruction (MPKI) for the LLC. The results are shown in Fig. 6d. The MPKI error is calculated in terms of absolute values, $e = |mpki_{stat} - mpki_{ca}|$, where $mpki_{ca}$ is the MPKI for cycle-accurate simulation. We show the average error grouped by the number of threads on the top chart and the plot error per correlation in the bottom figure. Similar to the LLC miss rate error, the MPKI error increases a lot for *NSM* as the number of threads increases, from $0.2MPKI$ to $5.5MPKI$. As for *Shalom* and *Condens*, the errors only range from $0.2MPKI$ to $1.1MPKI$, and from $0.5MPKI$ to $1.4MPKI$, respectively.

Note that while *Shalom* and *Condens* improve the accuracy, they do not eliminate all sources of error for the LLC misses. There are a few reasons for this. One reason is the inability to capture the actual interleaving of threads, which might affect both the sequential sharing and the concurrent sharing. Another source of error is that during profiling we need to limit the history buffer size of the recently accessed items. This affects the RD calculation by creating false cold misses. One last reason is that during address generations, we randomly generate the RD. Therefore, the original sequence of RD is not preserved in the synthetic trace [46]. This can cause extra misses due to conflict misses and invalidations.

7.3 Coherence Accuracy

An important aspect of our sharing locality model is that it is able to reproduce with some level of fidelity the coherence state in the system. Fig. 7 shows the frequency of the cache line state for each access to the L3 cache. We show the average frequency for the benchmarks grouped by the number of threads, ranging from 1 to 16. In this figure, the CA results are the golden values.

As we can observe, the traditional *NSM* is not able to capture the trends of the coherence states for applications with more than one thread. In *NSM* there is an increased occurrence of the invalid state. For instance, for benchmarks with 16 threads *NSM* reports 75% of the accesses to be on cache lines whose states are invalid, when the actual fraction is 17% reported by the CA simulation. Our proposed sharing locality model helps to preserve the correlation between the state transitions, as we can see by *Shalom* and *Condens*. However, here we observe some subtle inversion. Both, *Shalom* and *Condens* technique becomes more accurate when the number of threads increases.

We also observe that the frequency of the state *shared* is over-represented in all three models. Obtaining a perfect representation of the cache line states is challenging. As mentioned previously, the sequence of the RD generated during the synthesis highly affects how each of the cache lines is accessed which can cause transitions that do not occur in the CA simulation. Another factor that contributes to inaccuracy for the states is the modeling for the type of request (read/write). During profiling and synthesis, we assume that each cache line keeps three states, clean, dirty and new, and the transitions between them are modeled as a Markov Chain, similar to the approach in [23]. We believe that there could be better models to represent the transition between the states.

7.4 Speed-Accuracy Trade-Off

In this section, we show how *Condens* and *SDS* can be used for a trade-off between speed and accuracy. We use our sharing locality model (*Shalom*) in both simulation modes.

Fig. 8 shows the execution time accuracy and speedup for each of the models. The accuracy and speedup values are relative to those of the full-length CA simulation. The input for each of the models is shown in the *x*-axes. For *SDS* the input is the reduction factor (i.e., how much the trace should be reduced). For *Condens* the input is the convergence level. Higher values mean lower convergence. The continuous lines show the average over all data-points while the dashed lines show the average over the top 20% and the bottom 20% of the data-points, respectively. Note that when we set a large reduction factor or convergence level, the error also becomes extremely high. It is not intention to use such a large reduction factor or convergence

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

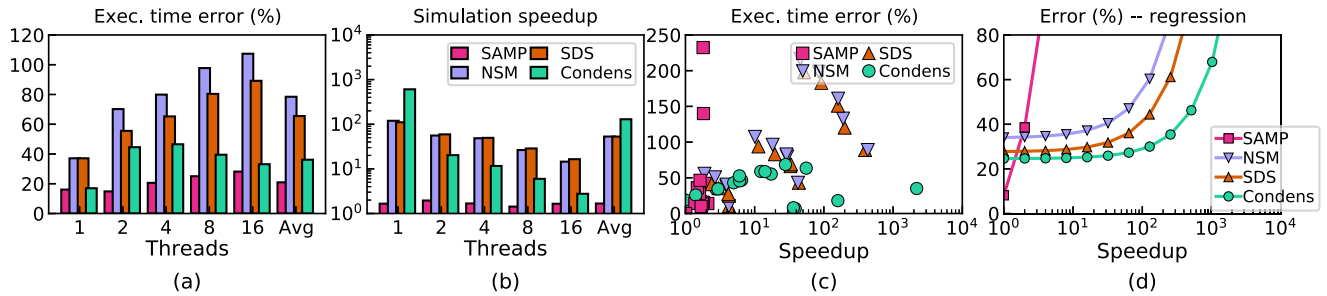


Fig. 9. Accuracy and speedup trade-offs for statistical and sampling simulation. The y-axis unit is shown on the top of the each subplot. (a) Average execution time error. (b) Average simulation speedup. (c) Scatter for average error per speedup. (d) Regression of average error per speedup, highlighting the speedup-saturation points.

level in practice, but rather, to give a comparison of the trends of the two techniques.

SDS has a more direct control over the simulation speed as it determines the trace-length to be simulated. We can observe a linear relationship between the reduction factor and the speedup in the bottom-left chart of Fig. 8. As the simulation length is reduced, the simulation error increases. Since *SDS* has no bounds on the simulation error, we observe a huge error variation between the simulation points with the same reduction factor. For instance, for a reduction factor of 100x, we notice an average simulation error of around 4% for the bottom 20% data-points, while the average error over the top 20% data-points is 242%. The high variation in the simulation error renders the *SDS* less attractive in some situations.

Our proposed technique, *Condens*, attempts to address this issue by giving the user better control over the error. The results in the upper-right chart show that the difference between the maximum error and minimum error is much smaller. Besides, this difference only changes slightly for a sweep in the convergence level from 0.1 to 100. However, in *Condens*, the simulation speed is not controlled directly by the user. This is observed in the bottom-right chart, where the maximum speedup obtained is about three orders of magnitude higher than minimum speedup for the same convergence level of 100.

7.5 Sampling Versus Statistical Simulation

Another method for speeding up simulation of computer systems is sampling simulations such as the SMARTS [1] and SimPoint [3]. These techniques were initially designed for single-core applications and need some adjustments to work properly in multithreaded applications. Ardestani *et al.* identified that the primary issue is that the traditional approaches sample instructions [4]. Therefore, they proposed sampling time to achieve sampling for multithread applications. We implement a technique similar to theirs in our simulation environment and compare it to the statistical methods. We refer to their technique as sampling (*SAMP*).

In *SAMP* simulation, we cycle through three phases iteratively: functional warmup (FW), detailed warmup (DW), and detailed simulation (DS). In FW, we only update the contents of the memory hierarchy and the state of the CPU registers. The timing does not progress during this stage. During DW, we simulate the system as in a standard CA simulation, including all timings and state transitions. However, we do not update the performance counters. The main idea is that doing otherwise leads to bias in the performance results. The last stage is the standard CA simulation where timing and performance counters are simulated normally. In a typical *SAMP*, the length of the FW is 4960K clock cycles, DW is 10K cycles, and the DS is 30K, leading to a period of 5M clock cycles.

To compare the methods, we sweep their input parameters obtaining four configurations for each approach. For both *NSM* and *SDS* we choose reduction factors of $\{1, 10, 10^2, 10^3\}$. For *Condens*, we set convergence levels of $\{0.1, 1, 10, 100\}$. In sampling, we fix DW and DS at the typical values and vary the period within the set {50K, 500K, 5M, 500M}. We simulate all 45 benchmarks for each configuration. In total, we obtain $45 \times 4 = 180$ simulation points per method. Fig. 9 summarizes the results.

To put things in perspective, we plot the error by the speedup in Fig. 9c. Values closer to the bottom-right are better. Each point is the average over benchmarks with the same count of threads and a specific simulation configuration. For instance, the right-most point is the average for single-threaded applications simulated using the *Condens* technique with convergence level 100. We can observe that *SDS* and *NSM* have many points with a high speedup (greater than 10x). However, the simulation error of each of them is high, greater than 50% for *SDS* and even higher for *NSM*. *SAMP*, on the other hand, has a high concentration of points with lower speedup and lower error. But also, *SAMP* has several points below speedup 3x and errors beyond 100%. This happens because after a certain stage, increasing the sampling rate in *SAMP* does not improve the simulation speed but continues to degrade accuracy. Therefore, we see a speedup saturation for the *SAMP* technique at around 3x. *Condens* is able to remain with low error rate while also obtaining a considerable number of points with high speedup.

To better visualize the trends we plot the trend line for each of the models in Fig. 9d. The trend line is obtained by fitting a second-degree polynomial curve. The fitting minimizes the squared error, $E = \sum_{i=0}^{N} |e_i - p(s_i)|^2$), where $p(s_i)$ is the polynomial function. During the fitting, we ignore the best and the worst 5% of the data-points for each model. It is not the intention of these trend-plots to predict the error/accuracy, but rather, to give a sense of the trends in the data.

As can be observed, all models reach a point of saturation. The sampling-based technique has an early saturation, at a speedup of 3, caused by the overheads of application emulation and functional simulation. The statistical simulations of *NSM* and *SDS* achieve the speedup bottleneck at much higher speedup, about two orders of magnitude

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

higher than sampling-based simulation. At low speedup levels, *Shalom* has better accuracy than *NSM*. However, *SDS* and *NSM* hit the speedup wall at about the same speedup, 10^2. The use of *Condens* simulation pushes the speedup wall slightly further, occurring at around 10^3. This is caused by the fact that *Condens* maximizes the speedup given a certain error level. Hence, applications that converge quicker to the average behavior achieve higher speedup.

Fig. 9a shows the average execution time error across all variations of parameters and grouped by the thread-count. We discard points beyond the saturation point as there is no gain in keeping them, considering the data points beyond the saturation point would only increase the error without gaining in speedup. As we can observe, for most of the techniques, the simulation error increases as the number of threads increases, with *Condens* being far less sensitive than the others. The main reason is that *Condens* monitors accuracy during simulation. All the other methods are open-loop and, therefore, leave the error unbounded. On average, the original statistical simulation disregarding sharing locality, *NSM*, has the highest error among all, over 70%. *Condens* is around 2x more accurate than the *NSM*, with an average error of 36%. *SAMP* is the most accurate, with an average error of 21%.

In Fig. 9b we report the average speedup across all variation of parameters. In general, the *SAMP* simulation is the slowest. This is caused by the fact that in sampling simulation we pay the price of emulating the application (QEMU overhead) and the price of realizing functional warmup. These overheads do not exist in statistical simulations. The average speedup across all threads is around 2x for *SAMP*, while the statistical counterparts have speedups higher than 50x. Among the statistical methods, *SDS* is the fastest, losing only for *Condens* for single-thread benchmarks. *Condens* is more sensitive to the number of threads regarding the speedup. This happens because multithread applications tend to have a longer time to reach the same level of convergence. The overall average speedup for *Condens* is around 130x. Note that, if needed, we could still increase the speedup of *Condens* by setting an even higher convergence level. This would come at the expense of increased error. The overall average speedup for *SAMP* is around 2x with an average error of 21%. *Condens* reaches a similar average error at around 10x speedup by averaging over the data-points under convergence levels of 0.1 and 1.

In summary, we believe that statistical simulation complements the sampling simulation in several ways. While, sampling techniques can achieve higher accuracy at low sampling rates, they are unable to achieve high speedup as their speedup saturation is low. Statistical techniques provide better flexibility to trade-offs between speed and accuracy, being able to achieve similar accuracy as sampling simulation at much higher speedup.

7.6 Limitations and Future Work

Real applications tend to exhibit a burst of cache misses or hits. One of the limitations of statistical simulations based on RD is that they do not capture these bursty behaviors. This happens because we often store the histogram distribution of RD rather than the sequence [47]. Therefore, during generation, we are only able to reproduce the average behavior. To reproduce bursts in statistical simulator we need to include extra profiling information. The burst modeling can be inspired by the SynFull technique [15], [48]. In SynFull, they classify epochs into macro-phases and profile the transitions between these phases. Each macro-phase represents a particular behavior with its characteristic. A similar idea could be used for statistical simulation based on RD. This lies outside the scope of this work so we leave it for future investigations.

One concern regards the thread interleaving during profiling. As mentioned previously, we assume uniform interleaving during profiling to mask the influence of the underlying architecture and obtain consistent profling results. However, the actual thread interleaving on the target architecture under evaluation may be different from the one used in profiling, resulting in misleading resource contention. As shown in Section 3.4, the impact of mismatched interleaving is expected to be low according to our experiments. Nonetheless, the influence will be exaggerated in simulating asymmetric multicore architectures [49] where threads map to different kind of cores. Other interleavings [50] may be deployed to tackle this issue. We leave the exploration of other interleavings to our future work.

Another concern regards the application variability. The application behavior can vary from different runs. Potential causes for variations are interference from other applications, OS overheads, and even non-deterministic IO behavior. Variability is especially higher for multi-threaded applications than single-thread [51], [52]. Currently, our profiling only covers a single run of the application and is not able to capture the variability. Therefore, it is possible that for applications with high variability our models would represent a biased instance. One potential solution is to profile the application for several runs recording the average and deviation for each metric. However, not all performance metrics are composable and computing their average might be not as simple as it sounds [53]. We leave as future work the evaluation of applications with high variabilities, such as server applications.

8 Related Work

To the best of our knowledge, this is the first work to introduce a sharing locality model for statistical simulation with profiling and synthesis algorithms.

Variations of RD Locality Models. We believe there is still room for improvement in our models. In our work, we used the general RD to model memory locality, but other methods could also be used. Some examples are STM [11], WEST [12], HRD [23], and HALO [24]. WEST and STM can capture the contention in the shared resources better than the traditional RD. HRD, on the other hand, can capture locality seen at multiple layers, e.g., locality of pages and of cache lines. However, WEST and HRD are not able to precisely capture the stride locality, rendering them unsuitable for studying prefetching. HALO attempts to resolve this issue. While STM can also capture prefetching behaviors, HALO has lower storage overhead. Currently, our framework is fully compatible with WEST and HRD. STM and HALO would require slightly more effort. The choice among these models should be assessed case by case, as each of them excels in one aspect. Another weakness of RD

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

is that it is not effective in modeling sophisticated replacement policies. In [54], the authors introduce models to cover replacement policies other than LRU.

RD Measurement Algorithms. Originally in [40], the authors adopt a stack data-structure to calculate RD with time complexity $O(NM)$ and space complexity $O(M)$, where N is the trace length and M is the number of unique addresses. Later, the time complexity is reduced to $O(NlogM)$ using tree-like data-structures [41], [42], [43]. Recently, approximation algorithms with time complexity $O(Nlog^2M)$ and space complexity $O(log(M))$ are proposed in [38], [45]. In [55], the authors target to reduce the storage overhead. Sampling [56] and parallelization [39], [57] can also be utilized to accelerate RD measurement. While our work use [43] to measure RD during profiling, other methods can also be potentially incorporated to reduce the cost of profiling. We leave it for future investigations.

Architecture-Independent Models. Obtaining memory locality for multithread applications with full architecture independence is a challenge. We understand that to achieve true independence we need to use composable models such as the ones discussed in [53]. Other models that can reduce the influence of architecture in the profiling are the private and the concurrent RD [29]. Reuse interval (RI) is another commonly used architecture-independent locality model. It is composable between independent programs if they are uniformly interleaved, and can also be used to compute miss ratio curve [58]. RI can be measured in $O(N)$ time for a trace length of N and converted to RD approximately [59]. However, it is still an open question whether these methods can be applied in statistical simulation of multithread applications.

Statistical Simulation for CPU Microarchitecture. RD is also not able to capture CPU microarchitecture effects, such instruction-level parallelism and memory-level parallelism. In our models, we ingnore instruction-level details, such as instruction dependency, but nothing prevents them from being used together with our models. One example is the performance cloning technique discussed in [60]. In this work, the authors propose profiling the registers dependencies as well as the basic-block flow to build statistical simulation for single-core applications. Other models that capture CPU microarchitecture effects are [61], [62], [63]. In [64], the authors propose a technique a modeling technique for memory-level parallelism.

Other Statistical Simulation Techniques. Similar statistical simulation methodologies targeting multiprogram applications are [25], [26]. In both works, they target multiprogram simulations and ignore data-sharing. Another approach is to realize statistical synthesis for the NoC such as in SynFull [15], [48]. In these works, the authors showed that it is possible to achieve very high accuracy in the traffic by profiling and generating traces at the NoC level. Their approach is also dependent on the architecture, requiring reprofiling for different architectures. In our work, we build statistical models at the application level, capturing the behavior of the application. Therefore, our models can be used to investigate complete systems.

9 Conclusion

In this work, we improve the traditional statistical simulation from two major aspects. The first improvement is the introduction of *Shalom*, a sharing-locality model that allows us to implement statistical simulation for shared memory applications. The second improvement is a technique that permits the simulator to monitor the statistical convergence of the synthesis, referred to as *Condens*, which gives the user a new tool for trading-off accuracy and simulation speed, allowing the user to bound the simulation error for fully-associative caches while obtaining the maximum speedup. Our sharing-locality model can achieve 2x higher execution time accuracy than simulations disregarding data sharing. In addition, we also show that the commonly used sampling-based simulation has a speedup saturation of about 3x. Using our locality models, the traditional *SDS* achieves a speedup saturation of 100x, and the newly proposed *Condens* reaches 1000x.

Acknowledgments

This work was done by the authors while they were with Microelectronics Thrust, Hong Kong University of Science and Technology (Guangzhou), and Department of Electronic and Computer Engineering, Hong Kong University of Science. We would like to thank the reviewers for their thoughtful feedback. Fan Jiang and Rafael K. V. Maeda are co-first authors.

References

[1] R. E. Wunderlich, T. F. Wenisch, B. Falsafi, and J. C. Hoe, "Smarts: Accelerating microarchitecture simulation via rigorous statistical sampling," in *Proc. 30th Annu. Int. Symp. Comput. Archit.*, 2003, pp. 84–95.

[2] T. F. Wenisch, R. E. Wunderlich, M. Ferdman, A. Ailamaki, B. Falsafi, and J. C. Hoe, "SimFlex: Statistical sampling of computer system simulation," *IEEE Micro*, vol. 26, pp. 18–31, Jul. 2006.

[3] G. Hamerly, E. Perelman, J. Lau, and B. Calder, "Simpoint 3.0: Faster and more flexible program phase analysis," *J. Instruction Level Parallelism*, vol. 7, no. 4, pp. 1–28, 2005.

[4] E. K. Ardestani and J. Renau, "ESESC: A fast multicore simulator using time-based sampling," in *Proc. IEEE 19th Int. Symp. High Perform. Comput. Archit.*, 2013, pp. 448–459.

[5] T. E. Carlson, W. Heirman, K. Van Craeynest, and L. Eeckhout, "Barrierpoint: Sampled simulation of multi-threaded applications," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2014, pp. 2–12.

[6] T. E. Carlson, W. Heirmant, and L. Eeckhout, "Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulation," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2011, pp. 1–12.

[7] D. Sanchez and C. Kozyrakis, "ZSim: Fast and accurate microarchitectural simulation of thousand-core systems," in *ACM SIGARCH Comput. Archit. News*, vol. 41, pp. 475–486, 2013.

[8] J. E. Miller *et al.*, "Graphite: A distributed parallel simulator for multicores," in *Proc. 16th Int. Symp. High-Perform. Comput. Archit.*, 2010, pp. 1–12.

[9] J. Wang *et al.*, "Manifold: A parallel simulation framework for multicore systems," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2014, pp. 106–115.

[10] G. Malhotra, R. Kalayappan, S. Goel, P. Aggarwal, A. Sagar, and S. R. Sarangi, "ParTejas: A parallel simulator for multicore processors," *ACM Trans. Model. Comput. Simul.*, vol. 27, no. 3, pp. 1–24, 2017.

[11] A. Awad and Y. Solihin, "STM: Cloning the spatial and temporal memory access behavior," in *Proc. IEEE 20th Int. Symp. High Perform. Comput. Archit.*, 2014, pp. 237–247.

[12] G. Balakrishnan and Y. Solihin, "WEST: Cloning data cache behavior using stochastic traces," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit.*, 2012, pp. 1–12.

[13] Y. Wang, G. Balakrishnan, and Y. Solihin, "MeToo: Stochastic modeling of memory traffic timing behavior," in *Proc. Int. Conf. Parallel Archit. Compilation*, 2015, pp. 457–467.

[14] R. Ubal, B. Jang, P. Mistry, D. Schaa, and D. Kaeli, "Multi2Sim: A simulation framework for CPU-GPU computing," in *Proc. 21st Int. Conf. Parallel Archit. Compilation Techn.*, 2012, pp. 335–344.

[15] M. Badr and N. E. Jerger, "SynFull: Synthetic traffic models capturing cache coherent behaviour," in *Proc. ACM/IEEE 41st Int. Symp. Comput. Archit.*, 2014, pp. 109–120.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.

[16] S. R. Sarangi, R. Kalayappan, P. Kallurkar, S. Goel, and E. Peter, "Tejas: A java based versatile micro-architectural simulator," in *Proc. 25th Int. Workshop Power Timing Model. Optim. Simul.*, 2015, pp. 47–54.
[17] S. Hong and H. Kim, "An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness," in *Proc. 36th Annu. Int. Symp. Comput. Archit.*, 2009, pp. 152–163.
[18] C. Nugteren, G. van den Braak, H. Corporaal, and H. Bal, "A detailed GPU cache model based on reuse distance theory," in *Proc. IEEE 20th Int. Symp. High Perform. Comput. Archit.*, 2014, pp. 37–48.
[19] G. E. Suh, S. Devadas, and L. Rudolph, "Analytical cache models with applications to cache partitioning," in *Proc. 15th Int. Conf. Supercomput.*, 2001, pp. 1–12.
[20] Y. Liang and T. Mitra, "An analytical approach for fast and accurate design space exploration of instruction caches," *ACM Trans. Embedded Comput. Syst.*, vol. 13, pp. 43:1–43:29, Dec. 2013.
[21] A. Joshi, L. Eeckhout, R. H. Bell, and L. John, "Performance cloning: A technique for disseminating proprietary applications as benchmarks," in *Proc. IEEE Int. Symp. Workload Characterization*, 2006, pp. 105–115.
[22] A. Joshi, L. Eeckhout, and L. John, "The return of synthetic benchmarks," in *SPEC Benchmark Workshop*, pp. 1–11, 2008.
[23] R. K. V. Maeda, Q. Cai, J. Xu, Z. Wang, and Z. Tian, "Fast and accurate exploration of multi-level caches using hierarchical reuse distance," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2017, pp. 145–156.
[24] R. Panda and L. K. John, "HALO: A hierarchical memory access locality modeling technique for memory system explorations," in *Proc. Int. Conf. Supercomput.*, 2018, pp. 118–128.
[25] D. Genbrugge and L. Eeckhout, "Chip multiprocessor design space exploration through statistical simulation," *IEEE Trans. Comput.*, vol. 58, pp. 1668–1681, Dec. 2009.
[26] D. Genbrugge and L. Eeckhout, "Statistical simulation of chip multiprocessors running multi-program workloads," in *Proc. 25th Int. Conf. Comput. Des.*, 2007, pp. 464–471.
[27] X. Xiang, B. Bao, C. Ding, and Y. Gao, "Linear-time modeling of program working set in shared cache," in *Proc. Int. Conf. Parallel Archit. Compilation Techn.*, 2011, pp. 350–360.
[28] D. Chandra, F. Guo, S. Kim, and Y. Solihin, "Predicting inter-thread cache contention on a chip multi-processor architecture," in *Proc. 11th Int. Symp. High-Perform. Comput. Archit.*, 2005, pp. 340–351.
[29] M.-J. Wu, M. Zhao, and D. Yeung, "Studying multicore processor scaling via reuse distance analysis," *ACM SIGARCH Comput. Archit. News*, vol. 41, no. 3, pp. 499–510, 2013.
[30] J. J. Yi and D. J. Lilja, "Simulation of computer architectures: Simulators, benchmarks, methodologies, and recommendations," *IEEE Trans. Comput.*, vol. 55, pp. 268–280, Mar. 2006.
[31] Q. Guo, T. Chen, Y. Chen, and F. Franchetti, "Accelerating architectural simulation via statistical techniques: A survey," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, pp. 433–446, Mar. 2016.
[32] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proc. Conf. USENIX Annu. Tech.*, 2005, pp. 41–46.
[33] M. M. K. Martin *et al.*, "Multifacet's general execution-driven multiprocessor simulator (GEMS) toolset," *SIGARCH Comput. Archit. News*, vol. 33, pp. 92–99, Nov. 2005.
[34] R. K. V. Maeda *et al.*, "Jade: A heterogeneous multiprocessor system simulation platform using recorded and statistical application models," in *Proc. 1st Int. Workshop Adv. Interconnect Solutions Technol. Emerg. Comput. Syst.*, 2016, pp. 8:1–8:6.
[35] C. Sakalis, C. Leonardsson, S. Kaxiras, and A. Ros, "Splash-3: A properly synchronized benchmark suite for contemporary research," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2016, pp. 101–111.
[36] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: Characterization and architectural implications," in *Proc. 17th Int. Conf. Parallel Archit. Compilation Techn.*, 2008, pp. 72–81.
[37] H. Jin, M. Frumkin, and J. Yan, "The OpenMP implementation of NAS parallel benchmarks and its performance," NASA Ames Res. Center, Mountain View, California, USA, Tech. Rep. RTOP 509–10-31, 1999.
[38] Y. Zhong, X. Shen, and C. Ding, "Program locality analysis using reuse distance," *ACM Trans. Program. Lang. Syst.*, vol. 31, no. 6, pp. 1–39, 2009.
[39] Q. Niu, J. Dinan, Q. Lu, and P. Sadayappan, "PARDA: A fast parallel reuse distance analysis algorithm," in *Proc. IEEE 26th Int. Parallel Distrib. Process. Symp.*, 2012, pp. 1284–1294.
[40] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation techniques for storage hierarchies," *IBM Syst. J.*, vol. 9, no. 2, pp. 78–117, 1970.
[41] F. Olken, "Efficient methods for calculating the success function of fixed-space replacement policies," Lawrence Berkeley Lab., Berkeley, CA, USA, Tech. Rep. LBL-12370, 1981.
[42] R. A. Sugumar, "Multi-configuration simulation algorithms for the evaluation of computer architecture designs," Ph.D. Thesis, Dept. Comput. Sci. Eng., Univ. Michigan, Ann Arbor, MI, USA, 1993.
[43] G. Almási, C. Caşcaval, and D. A. Padua, "Calculating stack distances efficiently," in *Proc. Workshop Memory Syst. Perform.*, 2002, pp. 37–43.
[44] C. CaBcaval and D. A. Padua, "Estimating cache misses and locality using stack distances," in *Proc. 17th Annu. Int. Conf. Supercomput.*, 2003, pp. 150–159.
[45] C. Ding and Y. Zhong, "Predicting whole-program locality through reuse distance analysis," *ACM SIGPLAN Notices*, vol. 38, no. 5, 2003, Art. no. 245.
[46] H. Luo, P. Li, and C. Ding, "Thread data sharing in cache: Theory and measurement," in *Proc. 22Nd ACM SIGPLAN Symp. Princ. Pract. Parallel Programm.*, 2017, pp. 103–115.
[47] L. Yuan, C. Ding, P. Denning, and Y. Zhang, "A measurement theory of locality," 2018, *arXiv:1802.01254*.
[48] J. Yin, O. Kayiran, M. Poremba, N. E. Jerger, and G. H. Loh, "Efficient synthetic traffic models for large, complex SoCs," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2016, pp. 297–308.
[49] R. Kumar, D. M. Tullsen, P. Ranganathan, N. P. Jouppi, and K. I. Farkas, "Single-ISA heterogeneous multi-core architectures for multithreaded workload performance," in *Proc. 31st Annu. Int. Symp. Comput. Archit.*, 2004, pp. 64–75.
[50] J. Brock, C. Ding, R. Lavaee, F. Liu, and L. Yuan, "Prediction and bounds on shared cache demand from memory access interleaving," *ACM SIGPLAN Notices*, vol. 53, no. 5, pp. 96–108, 2018.
[51] A. R. Alameldeen and D. A. Wood, "Variability in architectural simulations of multi-threaded workloads," in *Proc. 9th Int. Symp. High-Perform. Comput. Archit.*, 2003, pp. 7–18.
[52] C. Bienia and K. Li, *Benchmarking Modern Multiprocessors*. Princeton, NJ, USA: Princeton Univ. Press, 2011.
[53] C. Ding and T. Chilimbi, "A composable model for analyzing locality of multi-threaded programs," Microsoft, Tech. Rep. MSR-TR-2009–107, 2009.
[54] N. Beckmann and D. Sanchez, "Modeling cache performance beyond LRU," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2016, pp. 225–236.
[55] J. Wires, S. Ingram, Z. Drudi, N. J. Harvey, and A. Warfield, "Characterizing storage workloads with counter stacks," in *Proc. 11th USENIX Symp. Oper. Syst. Des. Implementation*, 2014, pp. 335–349.
[56] Y. Zhong and W. Chang, "Sampling-based program locality approximation," in *Proc. 7th Int. Symp. Memory Manage.*, 2008, pp. 91–100.
[57] D. L. Schuff, M. Kulkarni, and V. S. Pai, "Accelerating multicore reuse distance analysis with sampling and parallelization," in *Proc. 19th Int. Conf. Parallel Archit. Compilation Techn.*, 2010, pp. 53–64.
[58] L. Yuan, C. Ding, W. Smith, P. Denning, and Y. Zhang, "A relational theory of locality," *ACM Trans. Archit. Code Optim.*, vol. 16, no. 3, pp. 1–26, 2019.
[59] X. Shen, J. Shaw, B. Meeker, and C. Ding, "Locality approximation using time," *ACM SIGPLAN Notices*, vol. 42, no. 1, 2007, Art. no. 55.
[60] A. Joshi, L. Eeckhout, R. H. Bell, and L. John, "Performance cloning: A technique for disseminating proprietary applications as benchmarks," in *Proc. IEEE Int. Symp. Workload Characterization*, 2006, pp. 105–115.
[61] S. Nussbaum and J. E. Smith, "Modeling superscalar processors via statistical simulation," in *Proc. Int. Conf. Parallel Archit. Compilation Techn.*, 2001, pp. 15–24.
[62] S. Nussbaum and J. E. Smith, "Statistical simulation of symmetric multiprocessor systems," in *Proc. 35th Annu. Simul. Symp.*, 2002, pp. 89–97.
[63] T. S. Karkhanis and J. E. Smith, "A first-order superscalar processor model," in *Proc. 31st Annu. Int. Symp. Comput. Archit.*, 2004, pp. 338–349.
[64] S. V. den Steen and L. Eeckhout, "Modeling superscalar processor memory-level parallelism," *IEEE Comput. Archit. Lett.*, vol. 17, no. 1, pp. 9–12, Jan.–Jun. 2018.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.



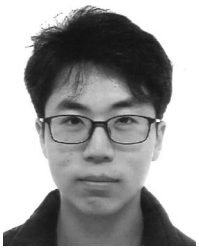
Fan Jiang (Student Member, IEEE) received the BEng degree in electronic information engineering from Zhejiang University, Hangzhou, China, in 2020. From 2020, he is currently working toward the PhD degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology (HKUST), Hong Kong. His research interests include modeling and simulation of multicore systems, program locality analysis, and hardware/software codesign.



Rafael K. V. Maeda (Student Member, IEEE) received PhD degree from the Hong Kong University of Science and Technology (HKUST), Hong Kong, in 2019. His research focuses on performance modeling of multicore processors. He is also interested in memory modeling, interconnect, CPU, and reconfigurable architectures.



Jun Feng (Student Member, IEEE) received the BS degree in electronic information science and technology from Nanjing University, Nanjing, China, in 2017. He is currently working toward the PhD degree with the department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong. His research interests include embedded system, multiprocessor systems, network-on-chip.



Shixi Chen (Student Member, IEEE) received the BEng degree in electronics and computer engineering with a second major in computer science from the Hong Kong University of Science and Technology (HKUST), Hong Kong, in 2018, where he currently working toward the PhD degree in electronics and computer engineering. His current research interests include opitcal interconnection networks, system-on-chip and optical power delivery system.



Lin Chen (Student Member, IEEE) received the BEng degree in integrated circuit design and integration system from the Huazhong University of Science and Technology, Wuhan, China in 2020. Since 2020, she is currently working toward the PhD degree with the Department of Electronic and Computer Engineering at Hong Kong University of Science and Technology (HKUST), Hong Kong. Her current research interests include power management of multi-core systems, machine learning and computer architecture.



Xiao Li (Student Member, IEEE) received the BE degree in electronic and information science and technology from Fudan University, Shanghai, China in 2019. From 2019, she is currently working toward the PhD degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology (HKUST), Hong Kong. Her current research interests include power delivery and management for multicore systems, machine learning, and hardware/software codesign.



Jiang Xu (Member, IEEE) received the PhD degree from Princeton University, Princeton, NJ, in 2007. From 2001 to 2002, he worked with Bell Labs, NJ, as a research associate. He was a research associate with NEC Laboratories America, NJ, from 2003 to 2005. He joined a startup company, Sandbridge Technologies, NY, from 2005 to 2007 and developed and implemented two generations of NoC-based ultra-low power MPSoC for mobile platforms. He is currently a professor and the acting department head of microelectronics thrust at Hong Kong University of Science and Technology. He is the founding director of Xilinx-HKUST Joint Lab and established Big Data System Lab and OPTICS (Optical/Photonic Technology for Interconnected Computing System) Lab. He currently serves as the associate editor for *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* and *IEEE Transaction on Very Large Scale Integration Systems*. He was an ACM distinguished speaker and an IEEE Computer Society distinguished visitor. He served on the steering committees, organizing committees and technical program committees of many international conferences. He authored more than 130 book chapters and papers in peer-reviewed journals and international conferences. His research interests include machine learning system, photonic-electronic codesign, MPSoC, power delivery and management, low-power embedded system, hardware/software codesign, interconnection network.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 15,2022 at 15:42:37 UTC from IEEE Xplore. Restrictions apply.